The Library Management System

Mr. Chey Somatra - Mr. Lin Powin - Mr. Keo Menglong - Ms. Mot Na
Mr. Nong Socheatra Mr. Eav Rotana - Ms. Hin Somphors
Mr. Kiry Ratanak - Mr. Srey Chanchhay

Abstract- Traditionally, library management has been conducted using manual methods such as paper-based records and conventional documentation systems. This approach has led to numerous challenges, including data loss, human errors, excessive time consumption, and difficulties in accurately tracking books, library members, and borrowing and returning records. With the continuous growth in the number of books and users, manual library management has proven to be inefficient and inadequate.

To overcome these limitations, a **Library Management System** was designed and developed. This system aims to automate essential library operations, including book catalog management, member registration, borrowing and returning processes, and data storage. By implementing a computerized system, data can be stored securely, accessed efficiently, and maintained with greater accuracy. Consequently, the Library Management System enhances operational efficiency, reduces errors, saves time, and provides a more reliable and modern solution for effective library management.

1. INTRODUCTION

A Library Management System (LMS) is software designed to help libraries manage their tasks more efficiently. It automates the organization of books, tracking of borrowed items, and management of users. By replacing manual processes, the system simplifies the work of librarians and improves service for library users. With an LMS, users can quickly search for books, reserve items, and access digital resources. The system also includes features such as fine tracking and the use of barcodes or RFID for greater efficiency. LMS is widely used in schools, colleges, public libraries, and organizations to make library operations faster, easier, and more accurate.

2. SYSTEM DESIGN(OOAD)

In software engineering, software design refers to the systematic structuring and organization of code, with a particular emphasis on extensibility and the ability to incorporate new features or modifications with minimal disruption to the existing system. In the context of a **Library Management System** (LMS), this implies that the system architecture should allow the addition of functionalities, such as e-book management or support for multiple library branches, without requiring a complete rewrite of the codebase.

Object-Oriented Analysis and Design (OOAD) provides a methodology for converting textual requirements into a well-defined software architecture using object-oriented models, including entities such as Book, Member, Loan, and Librarian. These models facilitate the consistent and efficient implementation of core functionalities, thereby enhancing the system's performance, scalability, and maintainability.

2.1 OOA(Object Oriented Analysis)

Object-Oriented Analysis (OOA) is the process of examining and understanding the problem domain by identifying real-world entities, their characteristics, and interactions. For the Library Management System, OOA transforms user requirements into a conceptual model that represents the library's operations through objects, their attributes, and behaviors.

2.1.1 Requirement

Member Management:

- Register new library members with personal details
- Update member information (name, email, phone number, address)
- Track member borrowing history
- Maintain member loan status (active/return)
- Delete members from the system when necessary

Book Management:

- Add new books to the library catalog
- Update book information (title, author)
- Track book availability status
- Manage total and available copies of each book
- Search books by various criteria (title, author, category)
- Remove books from the catalog

Loan Management:

- Process book borrowing requests
- Record loan dates and due dates
- Track overdue books
- Process book returns
- Maintain borrowing history

Librarian Operations:

- Librarian authentication and authorization
- Manage all member operations
- Manage all book operations
- Oversee loan transactions
- Generate reports on library activities
- View overdue loans and active loans

2.1.2 Attriute

Member Attributes

Attribute	Data Type	Description
id	Integer	Unique identifier for each member
name	String	Full name of the member
email	String	Email address for communication
phoneNumber	String	Contact phone number
address	String	Residential address
membershipStartDate	LocalDate	Date when membership started
isActive	Boolean	Current membership status
loans	List<Loan>	Collection of all loans associated with the member
isMember	Boolean	Flag indicating if person is a registered member

Book Attributes

Attribute	Data Type	Description
id	Integer	Unique identifier for each book
title	String	Title of the book
author	String	Author name(s)
publicationData	LocalDate	Date when book was published
category	String	Genre or classification of the book
publisher	String	Publishing company name
totalCopies	Integer	Total number of copies owned
availableCopies	Integer	Number of copies currently available

Loan Attributes

Attribute	Data Type	Description
id	Integer	Unique identifier for each loan transaction
loanDate	LocalDate	Date when book was borrowed
dueDate	LocalDate	Expected return date
returnDate	LocalDate	Actual return date (null if not returned)
member	Member	Reference to the member who borrowed
books	List<Book>	Collection of books in this loan
loanStatus	Enum	Status: ACTIVE, RETURNED, OVERDUE

Librarian Attributes

Attribute	Data Type	Description
id	Integer	Unique identifier for each librarian
name	String	Full name of the librarian
email	String	Email address (used for login)
phoneNumber	String	Contact phone number
managedBooks	List<Book>	Books managed by this librarian
isLibrarian	Boolean	Flag confirming librarian role

2.1.3 Behave

Member Behaviors

Query Operations:

- +getId(): Integer - Retrieve member's unique identifier
- +getName(): String - Get member's name

- +getEmail(): String - Get member's email
- +getPhoneNumber(): String - Get contact number
- +getAddress(): String - Get residential address
- +getLoans(): List<Loan> - Retrieve all loans for this member
- +getActiveLoans(): List<Loan> - Get currently active loans
- +getMembershipStartDate(): LocalDate - Get membership start date
- +isActive(): Boolean - Check if membership is active
- +hasOverdueLoans(): Boolean - Check if member has overdue books

Command Operations:

- +setId(id: Integer): void - Set member identifier
- +setName(name: String): void - Update member name
- +setEmail(email: String): void - Update email address
- +setPhoneNumber(phoneNumber: String): void - Update phone number
- +setAddress(address: String): void - Update address
- +setActive(isActive: Boolean): void - Activate/deactivate membership
- +addLoan(loan: Loan): void - Add a new loan to member's history
- +setMembershipStartDate(date: LocalDate): void - Set membership start date

Book Behaviors

Query Operations:

- +getId(): Integer - Get book's unique identifier
- +getTitle(): String - Get book title
- +getAuthor(): String - Get author name
- +getCategory(): String - Get book category
- +getTotalCopies(): Integer - Get total copies owned

- +getAvailableCopies(): Integer - Get currently available copies
- +getPublicationData(): LocalDate - Get publication date
- +getPublisher(): String - Get publisher name
- +isAvailable(): Boolean - Check if any copy is available for borrowing

Command Operations:

- +setId(id: Integer): void - Set book identifier
- +setTitle(title: String): void - Update book title
- +setAuthor(author: String): void - Update author
- +setCategory(category: String): void - Update category
- +setPublisher(publisher: String): void - Update publisher
- +setTotalCopies(copies: Integer): void - Set total copies
- +setAvailableCopies(copies: Integer): void - Update available copies
- +decreaseAvailableCopies(): void - Decrease when book is borrowed
- +increaseAvailableCopies(): void - Increase when book is returned

Loan Behaviors

Query Operations:

- +getId(): Integer - Get loan identifier
- +getLoanDate(): LocalDate - Get borrowing date
- +getDueDate(): LocalDate - Get due date
- +getReturnDate(): LocalDate - Get actual return date
- +getMember(): Member - Get associated member
- +getBooks(): List<Book> - Get list of borrowed books
- +getLoanStatus(): Enum - Get current loan status
- +isOverdue(): Boolean - Check if loan is overdue

- +isActive(): Boolean - Check if loan is currently active
- +getDaysOverdue(): Integer - Calculate days past due date

Command Operations:

- +setId(id: Integer): void - Set loan identifier
- +setLoanDate(date: LocalDate): void - Set borrowing date
- +setDueDate(date: LocalDate): void - Set due date
- +setReturnDate(date: LocalDate): void - Record return date
- +setMember(member: Member): void - Associate member
- +addBook(book: Book): void - Add book to loan
- +setLoanStatus(status: Enum): void - Update loan status
- +completeLoan(): void - Mark loan as returned

Librarian Behaviors

Query Operations:

- +getId(): Integer - Get librarian identifier
- +getName(): String - Get librarian name
- +getEmail(): String - Get email address
- +getPhoneNumber(): String - Get phone number
- +getManagedBooks(): List<Book> - Get books under management
- +isLibrarian(): Boolean - Verify librarian status

Command Operations:

- +setId(id: Integer): void - Set librarian identifier
- +setName(name: String): void - Update name
- +setEmail(email: String): void - Update email
- +setPhoneNumber(phoneNumber: String): void - Update phone

- +addManagedBook(book: Book): void - Add book to managed collection
- +removeManagedBook(book: Book): void - Remove book from management

2.2 OOD (Object-Oriented Design)

Object-Oriented Design (OOD) transforms the analysis models into a detailed software architecture. It defines how objects will be implemented as classes, establishes relationships between classes, and specifies the interfaces through which objects interact.

2.2.1 Entity Classes

Member Class

```
public class Member {
    private Integer id;
    private String name;
    private String email;
    private String password;
    private String phoneNumber;
    private String address;
    private LocalDate membershipDate;
    private Boolean isActive;
    private Boolean isMember;
    private List<Loan> loans = new ArrayList<>();
}
```

Book Class

```
public class Book {

   private Integer id;
   private String title;
   private String author;
   private LocalDate publicationDate;
   private String category;
   private Integer totalPages;
   private Integer totalCopies;
   private Integer availableCopies;
   private Boolean isAvailable;
}
```

Loan Class

```
public class Loan {
    private Integer id;
    private LocalDate loanDate;
    private LocalDate dueDate;
    private LocalDate returnDate;
    private LoanStatus status;
    private Member member;
    private List<Book> books = new ArrayList<>();
    public enum LoanStatus {
        ACTIVE,
        RETURNED,
        OVERDUE
    }
}
```

Librarian Class

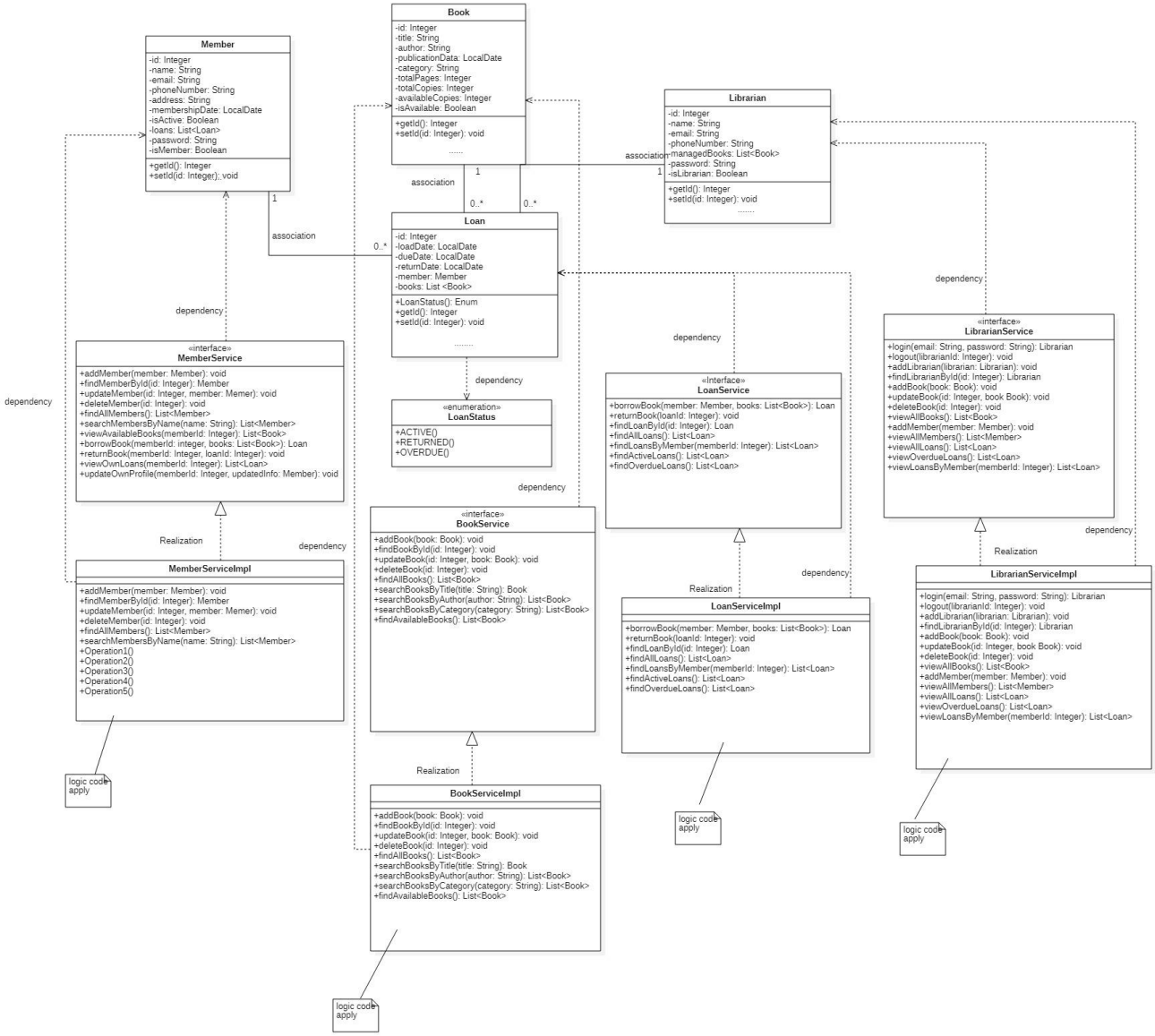
```
public class Librarian {
    private Integer id;
    private String name;
    private String email;
    private String password;
    private String phoneNumber;
    private Boolean isLibrarian;
    private List<Book> managedBooks = new ArrayList<>();
}
```

2.2.2 Relationship

Relationship

Relationship	Type	Multiplicity	Purpose
Member → Loan	Association	1 to 0..*	Track borrowing history
Loan → Book	Association	1 to *	Manage borrowed books
Librarian → Book	Association	1 to 0..*	Administrative management
Services → Entities	Dependency	-	Business operations
LoanService → BookService	Dependency	-	Update availability
LoanService → LoanStatus	Dependency	-	Status management
LibrarianService → All Services	Dependency	-	Unified access
Interfaces → Implementations	Realization	-	Contract fulfillment

Diagram



3. Conclusion

The Library Management System demonstrates a well-architected Object-Oriented design that effectively addresses the challenges of traditional manual library management. Through systematic application of Object-Oriented Analysis and Design (OOAD) principles, the system transforms complex library operations into a maintainable, scalable, and efficient software solution.